

A scalable universal Ising machine based on interaction-centric storage and compute-in-memory

Received: 1 May 2023

Accepted: 15 July 2024

Published online: 13 August 2024

 Check for updates

Wenshuo Yue^{1,2,7}, Teng Zhang^{1,7}, Zhaokun Jing¹, Kai Wu³, Yuxiang Yang¹, Zhen Yang¹, Yongqin Wu⁴, Weihai Bu⁴, Kai Zheng⁴, Jin Kang⁴, Yibo Lin¹, Yaoyu Tao², Bonan Yan^{1,2}✉, Ru Huang^{1,2} & Yuchao Yang^{1,2,5,6}✉

Ising machines are annealing processors that can solve combinatorial optimization problems via the physical evolution of the corresponding Ising graphs. Such machines are, however, typically restricted to solving problems with certain kinds of graph topology because the spin location and connections are fixed. Here, we report a universal Ising machine that supports arbitrary Ising graph topology with reasonable hardware resources using a coarse-grained compressed sparse row method to compress and store sparse Ising graph adjacency matrices. The approach, which we term interaction-centric storage, is suitable for any kind of Ising graph and reduces the memory scaling cost. We experimentally implement the Ising machine using compute-in-memory hardware based on a 40 nm resistive random-access memory arrays. We use the machine to solve max-cut and graph colouring problems, with the latter showing a 442–1,450 factor improvement in speed and 4.1×10^5 – 6.0×10^5 factor reduction in energy consumption compared to a general-purpose graphics processing unit. We also use our Ising machine to solve a realistic electronic design automation problem—multiple patterning lithography layout decomposition—with 390–65,550 times speedup compared to the integer linear programming algorithm on a typical central processing unit.

Combinatorial optimization problems (COPs) search for an optimal solution among a massive number of possible combinations of discrete parameters, such as the travelling salesman problem, graph maximum cut (max-cut) and graph colouring^{1,2}. COPs appear in various applications, including traffic planning, DNA sequencing and electronic design automation (EDA)³. However, they often have non-polynomial time complexity and are challenging to solve⁴. The large space of multiple parameters means that finding an optimum solution using brute-force

searching is not feasible. Although the absolute optimal solution might not be achievable, heuristic algorithms have been developed to find ‘approximate optimal’ solutions that can be useful in practical cases^{5,6}.

An Ising machine is a computer that can perform such a heuristic process by mimicking the evolution of a physical system with spin–spin coupling^{7,8} (see Supplementary Note 1 for the mathematical principles of Ising machines). By solving the ground state of a given physical system, Ising machines map COPs onto Ising models

¹Beijing Advanced Innovation Center for Integrated Circuits, School of Integrated Circuits, Peking University, Beijing, China. ²Institute for Artificial Intelligence, Peking University, Beijing, China. ³College of Electron and Information Engineering, Hebei University, Baoding, China. ⁴Semiconductor Technology Innovation Center (Beijing) Corporation, Beijing, China. ⁵Guangdong Provincial Key Laboratory of In-Memory Computing Chips, School of Electronic and Computer Engineering, Peking University, Shenzhen, China. ⁶Center for Brain Inspired Intelligence, Chinese Institute for Brain Research (CIBR), Beijing, China. ⁷These authors contributed equally: Wenshuo Yue, Teng Zhang. ✉e-mail: bonanyan@pku.edu.cn; yuchaoyang@pku.edu.cn

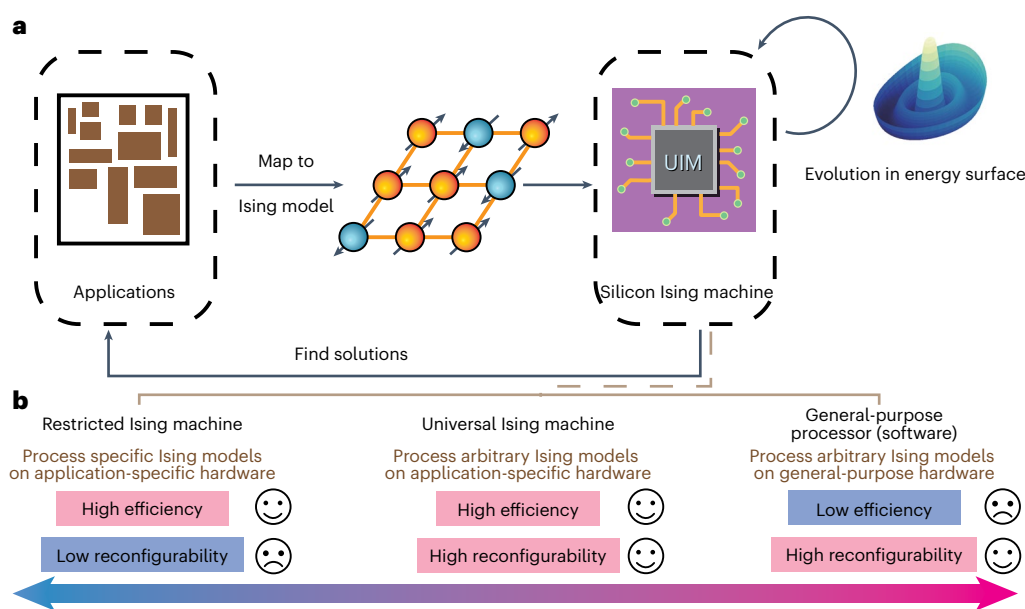


Fig. 1 | Workflow and classification of Ising machine. **a**, Practical NP-hard COPs can be extracted to the Ising model and then deployed on the Ising machine. A silicon Ising machine emulates the behaviour of the Ising model. The state of the Ising system evolves in its energy surface and finally converges to its optimal point (minimum). The evolution result presents a solution for the

practical problem. **b**, The spectrum of Ising machines with different levels of reconfigurability and computing efficiency. The proposed UIM lies at the balance point of these considerations, which achieves high efficiency while maintaining computing generality.

with Ising graphs (Fig. 1a). The vertices and edges of an Ising graph represent the spin states and the coupling interaction, respectively. Recently, quantum computers and silicon-built annealing machines have increased the speed of Ising machines^{9–13}. Silicon-built annealing machines are hardware processors that emulate the physical process through calculations. Taking advantage of deep-submicrometre very large-scale integration technology, this approach can build Ising machines that offer low cost and miniaturization, and can function at room temperature^{14–21}.

Silicon Ising machines have been implemented using complementary metal-oxide-semiconductor (CMOS) technology as application-specific integrated circuits (ASICs), field-programmable gate arrays (FPGAs)^{7,22,23} and software on general-purpose processors^{24,25}. These Ising machines compute the graph aggregation result of every vertex during each epoch to emulate the physical evolving process. Most existing hardware can reduce the computing time per epoch to microseconds or less. However, they are confined to limited Ising graph topology on ASICs (which are less flexible) or need better scalability on general-purpose processors (which are less efficient).

The key to tackling this area-performance scalability challenge is the mapping between Ising graphs and hardware. In Ising ASICs designed for efficiency, each vertex in an Ising graph (that is, a spin) maps to a single circuit module on hardware, and communication among these vertex circuits represents the edges of the Ising graph. Spin–spin interactions are simulated via the repetitive exchange of signals among vertex circuits. This is defined as spin-centric mapping. Communication (for coupling interaction) and synchronization among spins causes the overall compute speed to decrease, especially when the edges between the spins are sparse and irregular.

More seriously, this method is only valid for limited topological shapes of Ising graphs due to the limited on-chip routing resources, which is why these are sometimes referred to as restricted Ising machines. Restricted Ising machines are a class of Ising machines whose spin locations and connections are physically fixed with dedicated integrated circuits. They can only handle specific spin-connecting topologies, such as King’s graph and the Chimaera graph^{18,23}. By contrast, an

ideal Ising machine should be able to handle arbitrary connections of graphic vertices (Fig. 1b). A typical theoretical example of a universal Ising machine (UIM) with the highest flexibility is Ising machines with all-to-all connections (meaning each vertex in the Ising graph can connect to all other vertices). However, because of the difficulty routing these schemes, UIMs are usually too expensive as realistic COP solvers (see Supplementary Note 2 for a detailed comparison).

In this Article, we report a silicon UIM with reduced chip area and high reconfigurability. We propose an interaction-centric mapping, where the Ising graph is mapped onto the Ising machine by its adjacency matrices. Annealing processes are then simulated by processing the Ising graph’s adjacency matrix and the spin state vectors of the system. This contrasts with spin-centric Ising machines, where vertex circuits must be augmented by adding complex and reconfigurable routing channels. We develop a coarse-grained compressed sparse row (CG-CSR) method to efficiently compress and store sparse Ising graph adjacency matrices.

Furthermore, to eliminate repetitive data transfer, compute-in-memory (CIM) hardware is used as the core units^{26,27}. As optimization problems become sparser and more irregular, the large number of stored zeros exceed the hardware capabilities of CIM cores, but the CG-CSR compression stores only the non-zero submatrices of the adjacency matrix, supporting the mapping of larger problems within the same hardware area. We implement the proposed CIM-based UIM with a 128 × 128 one-transistor-one-resistor (1T1R) resistive random-access memory (RRAM) macro made with a 40 nm CMOS process. This is used to solve max-cut and graph colouring problems as representative binary-value and multi-level weight value COPs. We also solve a real-world EDA COP problem—multiple patterning lithography (MPL) layout decomposition—which is essential for sub-45 nm CMOS technologies, with a 390–65,550 times speedup towards convergence.

Interaction-centric UIM

The efficiency and flexibility of Ising machines are rooted in how they accommodate Ising graphs. In the conventional spin-centric design methodology, engineers customize processing elements for each

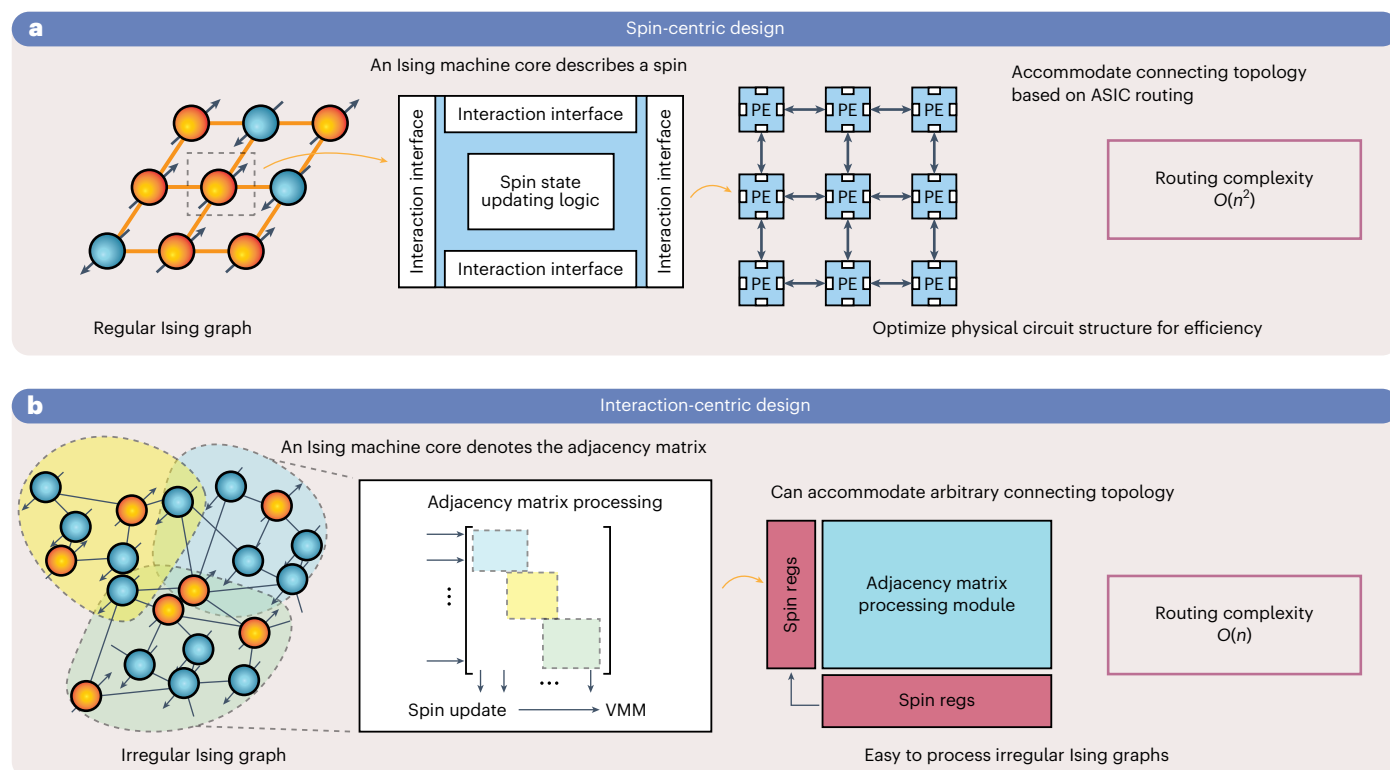


Fig. 2 | Comparison between spin-centric design and interaction-centric design. a, The spin-centric design usually uses a processing unit (PE) to represent a spin in the Ising graph. The connecting interfaces between PEs represent the interactions between spins. The routing complexity of spin-centric design is $O(n^2)$ to reach an ideal all-to-all connection. This indicates that the space complexity required for routing on a chip is proportional to the square of the edge number. It is usually hard for spin-centric design to process irregular

Ising graphs. **b**, The interaction-centric design develops an adjacency matrix processing module to store and process the interactions of the Ising graph. The spins are stored in peripheral registers. The routing complexity of interaction-centric design is $O(n)$ to reach an ideal all-to-all connection, which is simpler than spin-centric design. Benefitting from this simplicity, it is easy for an interaction-centric design to process irregular Ising graphs.

vertex in Ising graphs (Fig. 2a). The calculation of spin–spin coupling goes through physical routing buses between those processing elements. To support arbitrary Ising graphs, the spatial routing complexity, defined by the number of routing channels needed for all-to-all spin connection, has to be $O(n^2)$, where n is the number of spins involved (see Supplementary Note 23 for the complexity derivation). By contrast, the interaction-centric methodology leads to Ising graphs' hardware processing the adjacency matrices (Fig. 2b). The spatial routing complexity is hence $O(n)$ because the spins are lined up as a spin-status vector. Connections among processing elements (PEs) only exchange spin-status vectors. With reduced routing complexity, interaction-centric can support arbitrary topological relationships between spins after such interaction-centric machines are fabricated as ASICs, providing the maximized flexibility for Ising models.

Real-world Ising graphs are often irregular and sparse. Directly processing dense adjacency matrices wastes energy and computing time processing mostly zeros. Using the conventional element-wise compression technique (Fig. 3a) causes difficulty in the parallelism improvement because of element-by-element operations. Hence, we develop a sparse graph storing strategy, CG-CSR, to store irregular graphs in regular memory modules while maintaining reasonable parallelism. CG-CSR expands the processed objects from scalar elements in the sparse adjacency matrices to the non-zero submatrices (this is where the name of 'coarse-grain' originates, illustrated in Fig. 3b).

CG-CSR is designed to suit highly parallel vector-matrix multiplication (VMM) operation and can achieve a sufficient compression rate. Figure 3c illustrates the operating principle of CG-CSR to encode and store a practical sparse Ising graph efficiently. First, the adjacency matrix of a sparse Ising graph is partitioned into submatrices with

user-defined dimensions. Only non-zero submatrices are preserved, and their indices are stored in CSR format. In the example depicted in Fig. 3c, a 32×32 sparse adjacency matrix is partitioned into an array of 8×8 submatrices (each submatrix has 4×4 elements). Then, non-zero submatrices on the same row perform a 'shift-to-left' operation for compression. All-zero submatrices are abandoned. This remaining compressed matrix is stored in the memory and is referred to as 'the compressed array' in the following context. Note that the 'shift-to-left' process disturbs the column positions of each non-zero submatrix and so additional indices occupy a small space in the memory for future use in Ising computation. The indices include col and rowptr, encoding each non-zero submatrix's column and row positions (see Supplementary Note 4 for detailed definition and encoding examples of CG-CSR). Similar to conventional element-wise CSR format, the i th element in rowptr points to the col element above the row i in the block matrix. This technique achieves an average compress rate (that is, the original graph adjacency matrix ratio without any compressed representation over the number of scalar elements in the compressed array) by $8.42\times$ on cases in G-Set (Fig. 3d, see Supplementary Note 4 for the detailed configuration).

CG-CSR aims to bring convenience for exploiting parallel vector-matrix processing elements to quickly process meaningful (that is, non-zero) submatrices. Figure 3e shows the computation steps with Ising graphs stored in CG-CSR format. The evolution of the Ising machine updates the spin status, with a given Ising graph as the problem configuration. The primary computations are the spin-status vector multiplying the sparse Ising graph. To realize this, submatrices and the corresponding partial spin-status vector of epoch n should be applied to the VMM processing elements.

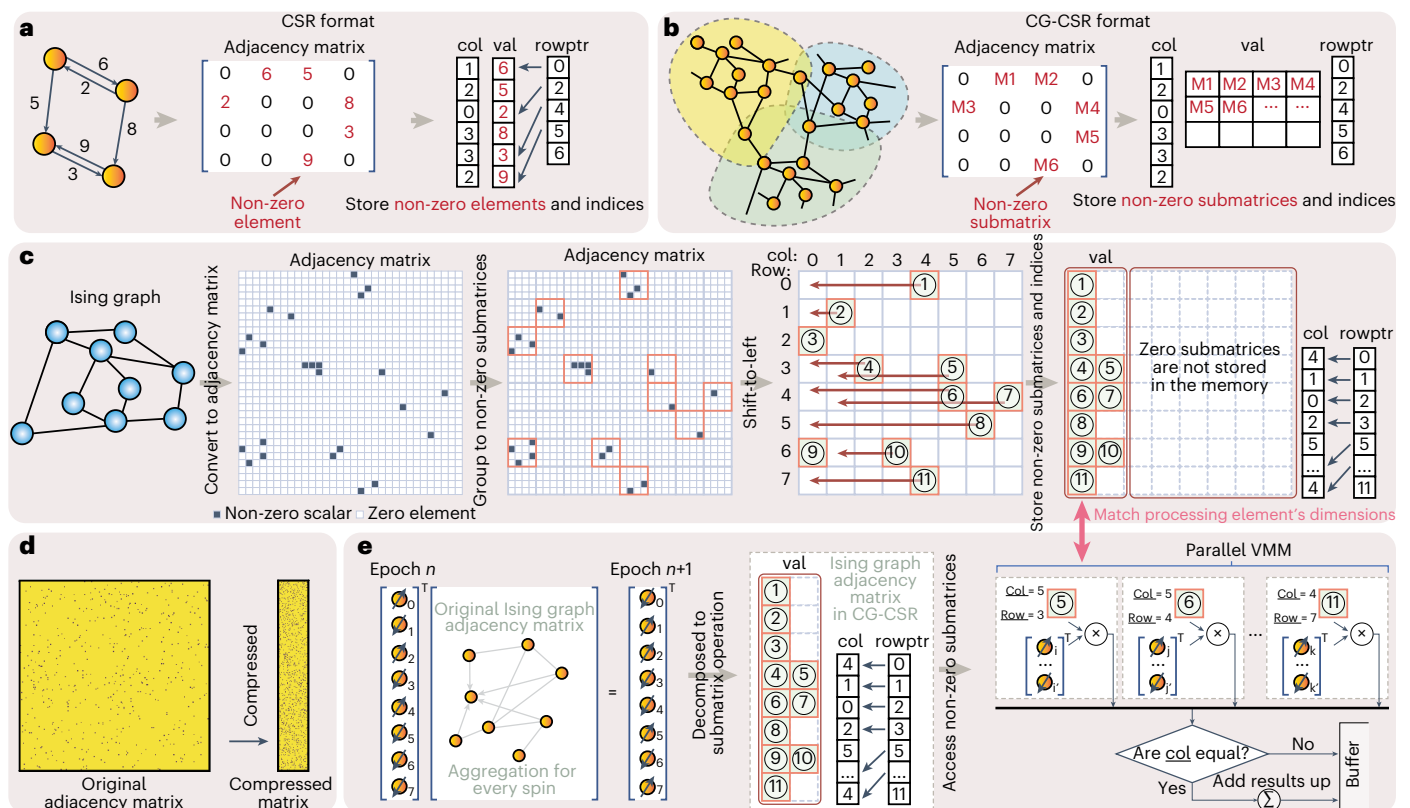


Fig. 3 | Illustration of the CG-CSR method and its application to Ising computing. **a**, Diagram of traditional CSR method. Instead of an adjacency matrix, CSR uses col, val and rowptr to store only non-zero elements. **b**, Diagram of coarse-grained-CSR method. This method stores non-zero submatrices (M1, M2 and so on) instead of non-zero elements. The non-zero submatrices are stored in the CIM module. **c**, CG-CSR maps an irregular Ising graph to a memory module in a rectangle shape. It selects non-zero submatrices from the adjacency matrix and compresses them together with a shift-to-the-left operation. After compression, only the non-zero submatrices are stored in the memory. **d**, The

comparison between the original adjacency matrix (left) and the compressed matrix (right). The black dots represent non-zero elements, while the yellow background represents the zeros. The CG-CSR method greatly increases the storing data density versus the original sparse adjacency matrix. **e**, During Ising computing, the VMM operation between the spin vector and Ising graph adjacency matrix is decomposed to submatrix operation based on CG-CSR. The submatrices are processed in parallel, and their results are guided by col information. The word length of the compressed matrix matches the processing element's dimensions.

Simultaneously, the column and row positions of a certain non-zero submatrix, that is col and row, are decoded from the stored hardware-friendly col and rowptr indices in CSR format. The VMM can be performed in parallel, using all available VMM processing elements. The partial VMM products (in vector forms) should be summed if and only if their col indices are the same.

Hardware architecture of UIM

The interaction-centric UIM computes simulated annealing per epoch in four steps: (1) access non-zero submatrices in CG-CSR; (2) calculate system Hamiltonian (that is, energy); (3) update spin status and (4) update Hamiltonian. CIM technology macro features minimal adjacency matrix access and parallel VMM operations. Figure 4a illustrates the block diagram for the proposed UIM hardware. An RRAM CIM core and CSR index memory together store the Ising graph. The addressing module recovers the non-zero submatrices from the compressed array. Partial sum cells (PSCs) receive VMM results from CIM macro and calculate the intermediate Ising graph aggregation values. Binarized aggregation values go to the global buffer as the input spin states for the next epoch.

For in-memory computation, the submatrix partition introduced in CG-CSR exploits the computing parallelism of CIM²⁸. In CIM, the non-zero submatrices are directly used for in-place VMM for Ising graph aggregation. For every CIM computation clock cycle, CIM executes one row of submatrices (more details of the CIM parallelism can be found in Supplementary Note 7).

The shift-to-left operation in CG-CSR squeezes the non-zero submatrices to the left side in each row, resulting in disordered column indices. On the other hand, the graph aggregation (VMM of the Ising graph adjacency matrix) requires accumulating sums of the non-zero submatrices in the compressed array that were initially in the same column. Therefore, an addressing module is used to solve this column index disorder problem to obtain the final VMM results regarding the original sparse Ising graph adjacency matrix. The addressing module works simultaneously with the CIM module in a pipeline manner and does not induce additional computing latency. To rapidly compare (row and col) indices, the addressing module includes a content-addressable memory (CAM) array (see Supplementary Note 8 for the implementation details). A CAM searches a given word in its memory. If the expression matches, the CAM returns the target address. The CAM array in the addressing module consists of multiple CAMs. The number of CAMs is the same as the maximum number of submatrices in one row of the compressed array. Each CAM works for a single column of submatrices. All CAMs store the same list of PSC indices. For example, if there are 500 PSCs, the stored data in CAM will be {0, 1, 2, ..., 499}. If the word length of each term in CAM is 9 bit, for example, then the required CAM size is 500 × 9 bits. If the number of submatrices in each row of the compressed array is 40, there will be 40 CAMs, and the required capacity of the CAM array is reasonably small at ~180 Kb.

We implement the UIM on a 128 × 128 1T1R bitcell CIM array²⁹ (Fig. 4b). The RRAM device is integrated with a standard 40 nm CMOS

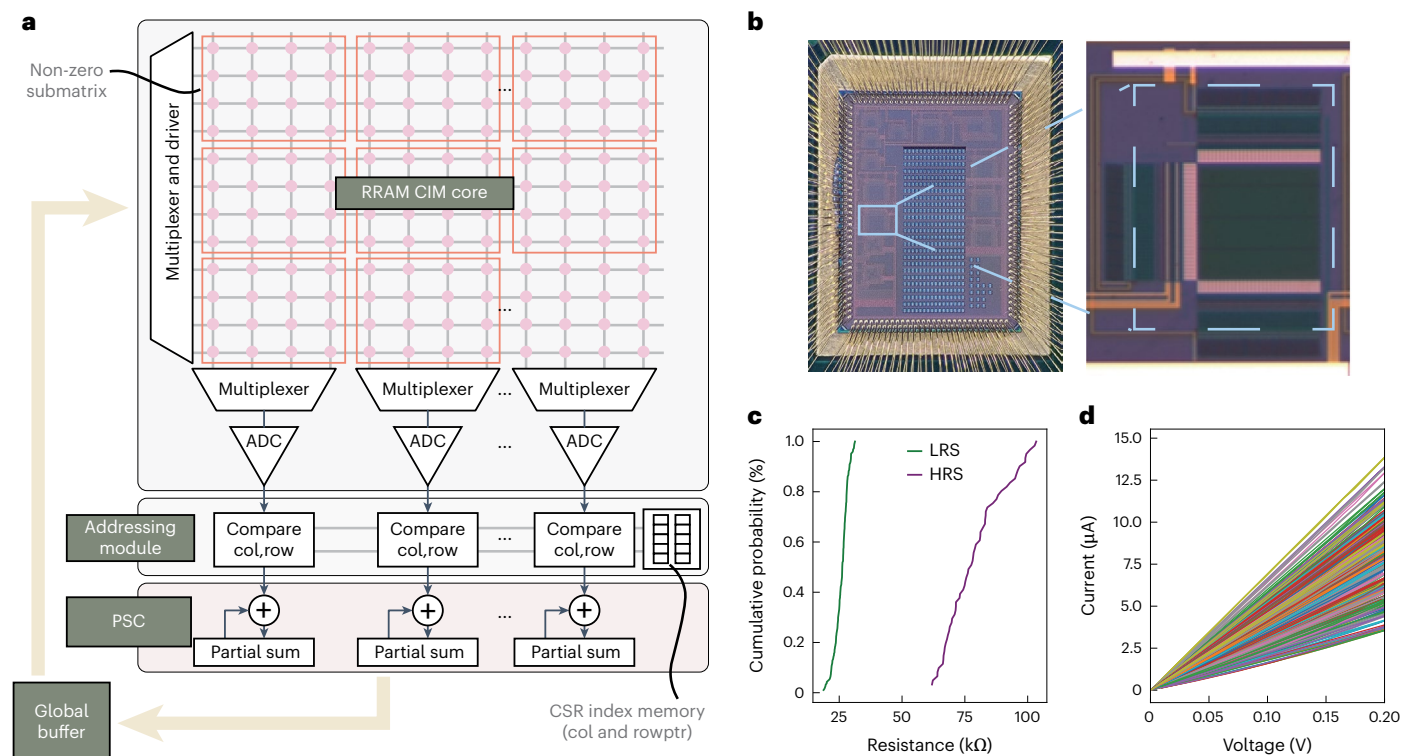


Fig. 4 | Illustration of implementing UIM on RRAM-based CIM module.

a, The non-zero submatrices are stored in RRAM CIM core. After the CIM results are read out by ADCs, the addressing module guides these results to PSCs, using the CSR indices stored in the addressing module. The spin values are updated inside PSCs and are further sent to the global buffer, where the spin values can

be used for the next computing epoch. **b**, Die photo of the fabricated RRAM chip. **c**, Statistics of RRAM conductance distribution in this work. The devices are operated by a regular forming, set and reset process. **d**, I - V linearity of the reading device's conductance. 'LRS' refers to low-resistance state and 'HRS' refers to high-resistance state.

process. The peripheral circuits decode the address and read memory bit cells. The macro can work in either memory mode or CIM mode. Figure 4b shows an optical microscope photo of a hybrid 1T1R crossbar array chip with each cell consisting of a transistor whose drain is connected to an RRAM cell to serve as the selector. The device follows steady bipolar switching characteristics after the forming process. Figure 4c shows the device's resistance distribution. The mean value of the low-resistance state remains at ~30 kΩ, which supports low-power operation. The device conductance shows high linearity while reading, ensuring high computing accuracy of the CIM (Fig. 4d).

The CIM macro is reconfigurable for different computing granularity by tuning the non-zero submatrix dimension (block size). Finer granularity, that is smaller block sizes, can eliminate more zeros and increase area efficiency. However, the degree of parallelism is lower in fine granularity, thus decreasing the computing throughput. Conversely, coarser granularity, which corresponds to larger block sizes, leads to high parallelism and speed, whereas adding more zeros in the compressed array lowers the memory utility rate. Therefore, a tradeoff in granularity exists between computing parallelism and memory efficiency. The appropriate block size for general cases in the used dataset is between 4×4 and 8×8 (Supplementary Note 22). The CIM macro can support the computation on different weight precisions, including binary weights and multi-bit weights (see Supplementary Note 25 for detailed discussions). The hardware cost of the proposed UIM relies on the number of spins. After a maximum scale is given, the proposed CIM-based UIM can solve all problems whose scales are less than the designed scale. As an example, we consider a chip architecture that can accommodate 2,000 spins. The primary design target is to execute any Ising models with $\leq 2,000$ spins and high sparsity.

COP solving with CIM-based UIM

This section develops a method to map and deploy typical COPs on the Ising model onto the proposed CIM-based UIM. Realistic non-deterministic polynomial-time (NP)-hard COPs can be categorized into binary-value COP (for example, graph max-cut problem^{7,13,14,19}) and multi-level-value COP (for example, graph colouring problem). Binary-value COP is common in literature^{7,8,13,14,19,30}, while multi-level-value COP lacks hardware acceleration because it cannot be directly mapped to a standard Ising model. In this section, we provide UIM solutions for both.

Binary-value COP refers to a COP where the value of each variable is binary. Since the spin value in the Ising model is also binary, binary-value COP naturally suits the standard Ising model. Considering the part of the Hamiltonian that is related to

$$H(\sigma_j) = -\sum_i J_{ij} \sigma_i \sigma_j - h_j \sigma_j = -\sigma_j \left(\sum_i J_{ij} \sigma_i + h_j \right) \quad (1)$$

where σ_i (or σ_j) is either +1 or -1 (binary), thereby the updating rule for arbitrary σ_i to minimize the Hamiltonian can be easily derived as:

$$\sigma_i^{\text{next epoch}} = \begin{cases} +1, & \sum_j J_{ij} \sigma_j + h_i > 0 \\ -1, & \sum_j J_{ij} \sigma_j + h_i \leq 0 \end{cases} \quad (2)$$

This equation reveals that the updated value of a spin depends on the aggregation of its connecting spins and the external field. The Ising machine hardware follows this rule to update the spins in each computing epoch.

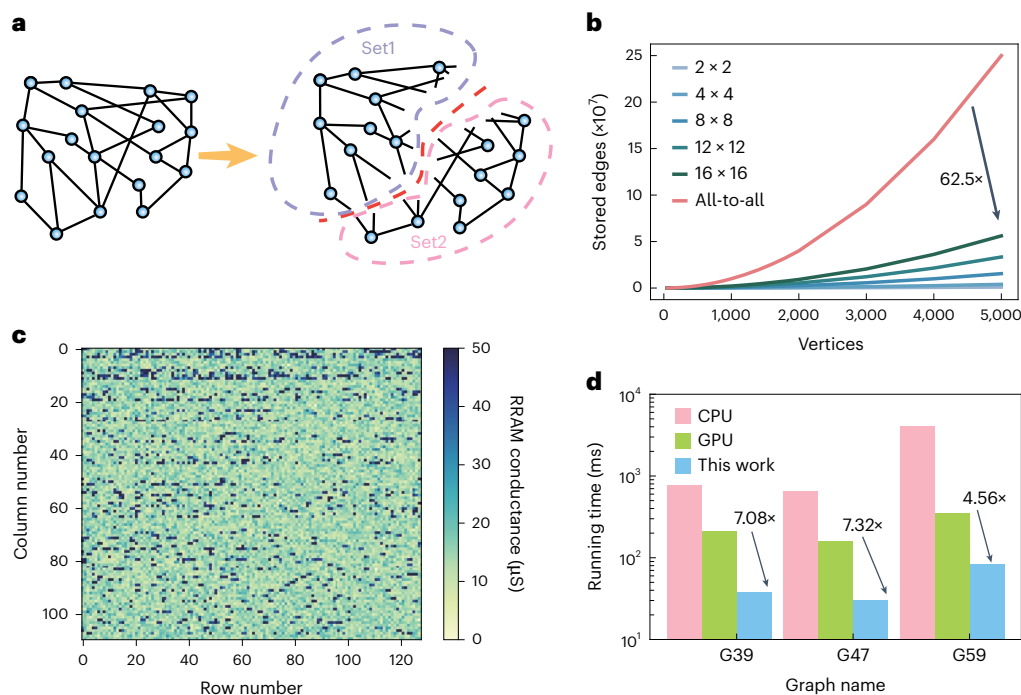


Fig. 5 | Max-cut demonstration. **a**, The goal of the max-cut problem is to find a division of the original graph that maximizes the sum of weights of cut edges between Set1 and Set2. **b**, Scaling of stored edges in the CG-CSR scheme. It only costs ~1% required edges in 4×4 CG-CSR, compared to the all-to-all connection. The memory saving is more prominent as the graph scale grows larger.

c, Measured conductance mapping of the compressed adjacency matrix on the RRAM CIM macro. **d**, Computing time of max-cut problems on proposed UIM, compared to CPU (Intel Xeon E5) and GPU (Nvidia Tesla K40c). This work computes roughly 4.56–7.32× faster than GPU.

As a typical binary COP, the max-cut problem is defined as given an undirected graph $G = (V, E)$ (V are the set of vertices and E is the set of edges), find a cut $C(G)$ for edges (a subset of edges) to maximize the sum of cut edges (Fig. 5a). It proves to be NP-hard and is within Karp's original NP-complete problems. The max-cut problem has broad applications, including circuit layout design and statistical physics^{31,32}.

To solve this problem in UIM, we first define that the 'weights' are $w_{i,j}$ for $i, j \in E$. Vertices are cut into V^+ and V^- . The set of cut edges is $\delta(W) = \{w_{i,j} | i \in V^+, j \in V^-\}$ and their corresponding vertices set is $\delta(V)$. The goal of max-cut problem is to maximize $C(G) = \sum_{w_{i,j} \in \delta(W)} w_{i,j}$. Then, let the vertices in V^+ represent spin +1 and the vertices in V^- represent spin -1. The problem is subsequently transformed to the Ising model. If the external field is not considered, the Hamiltonian will be:

$$\begin{aligned} H &= - \sum_{i,j \in V} J_{i,j} \sigma_i \sigma_j \\ &= - \sum_{i,j \in V^+} J_{i,j} \sigma_i \sigma_j - \sum_{i,j \in V^-} J_{i,j} \sigma_i \sigma_j - \sum_{i,j \in \delta(V)} J_{i,j} \sigma_i \sigma_j \\ &= - \sum_{i,j \in V^+} J_{i,j} - \sum_{i,j \in V^-} J_{i,j} + \sum_{i,j \in \delta(V)} J_{i,j} \\ &= - \sum_{i,j \in V} J_{i,j} + 2 \sum_{i,j \in \delta(V)} J_{i,j} \end{aligned} \quad (3)$$

Because the first term $\sum_{i,j \in V} J_{i,j}$ is a constant, H is mainly determined by the second term $\sum_{i,j \in \delta(V)} J_{i,j}$. We deploy the calculation of the Hamiltonian onto the Ising machine. As H evolves to its minimum, max-cut value $C_{\max}(G)$ of graph G is obtained. The optimization results for the max-cut problem are comparable to the best of non-zero traditional algorithms (Supplementary Note 15).

The CG-CSR method helps save memory to store the whole graph. As the graph size scales up, the advantage of CG-CSR becomes more prominent (Fig. 5b). CG-CSR with 16×16 submatrix could reduce memory cost to 62.5× smaller than the baseline of directly storing the

original adjacency matrix. The finer granularity costs less memory (see Supplementary Note 11 for general estimations). We experimentally map the compressed data on an RRAM CIM macro (Fig. 5c) and deploy them in the closed-loop CIM-based UIM system (Fig. 6). To compare its performance with central processing units (CPUs) and graphical processing units (GPUs), we consider the execution time for 1,000 epochs of simulated annealing algorithm in each hardware. Taking G59 as an example: our hardware takes 74.48 ms to complete, while the CPU and GPU take 4,070 and 340 ms, respectively. Our hardware reaches 4.56–7.32× speedup compared to GPU (Fig. 5d). The benchmark in this comparison is based on the Intel Xeon E5-2696v2 CPU processor and Nvidia Tesla K40c GPU.

Multi-level-value COP refers to problems in which the value of a variable is more than binary. We use the graph colouring problem as an example of multi-level-value COP. It aims to colour the vertices of a given graph with minimum colours such that no two neighbour vertices share the same colour³³. Conversely, an edge connecting two vertices with the same colour is called a 'conflict'. Efficiently solving graph colouring problems is highly beneficial because they widely occur in the computer-aided design of very large-scale chips, including applications such as layout decomposition³⁴, logic minimization³⁵ and register allocation³⁶.

The colour of each vertex is a multi-level variable, making the problem more complicated than the binary-value COP. Previous research has proposed an Ising model for the graph colouring problem. To consider whether a graph can be coloured with n colours, n spins are to represent a single vertex state. In other words, there are n copies of an Ising graph. The spin state in each graph is either +1 or 0. Each spin represents a possible colour for a vertex with a one-hot encoding scheme. In an ideal case after annealing convergence, only one spin remains +1 among the n -spin vertex, while the remaining $(n-1)$ spins are 0 (Fig. 6a). By doing this, the multi-level-value colouring problem collapses into an ensemble of binary-value Ising

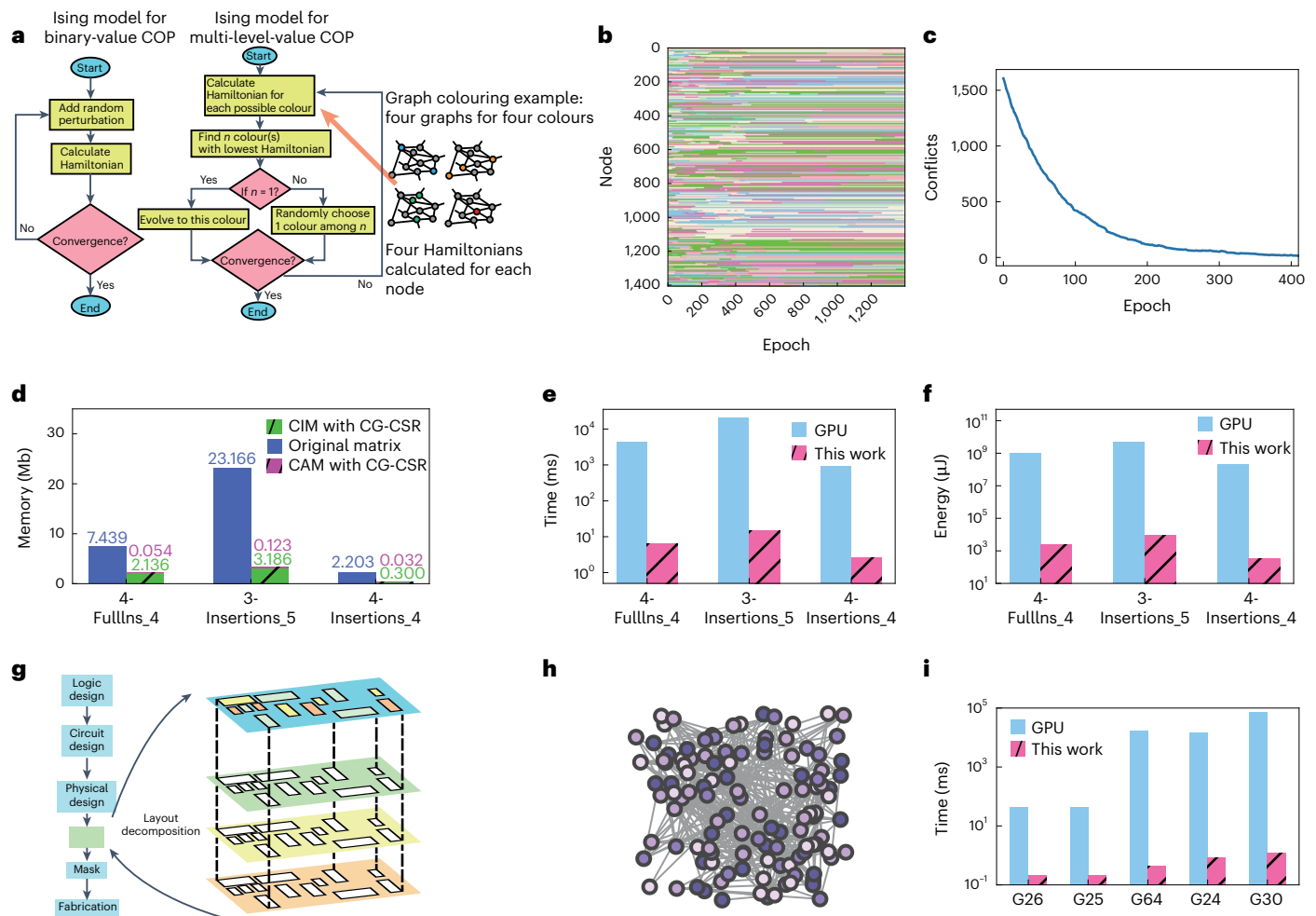


Fig. 6 | Graph colouring demonstration. **a**, Modified algorithm for mapping graph colouring problem to Ising machine. The modified algorithm duplicates the Ising graph to represent different colours. It uses the single-selecting rule to synchronize these graphs (see Supplementary Note 16 for the single-selecting rule). **b**, Colour evolution of all nodes in a 1,400-node graph. **c**, The evolution of conflicts over 3-Insertions_5 data. The system reaches convergence after about 400 epochs. **d**, CIM and CAM memory consumption compared to all-to-all connection scheme. **e**, Graph colouring time of UIM, compared to GPU (Nvidia

GeForce GTX480). **f**, Graph colouring energy consumption of UIM, compared to the same GPU. **g**, Layout decomposition is a practical graph colouring problem. Considering the layout of one layer in chip design as a graph, layout decomposition equals to colouring the graph with the least conflicts. **h**, The colouring result of an Ising graph extracted from layout decomposition problem in ISCAS dataset. **i**, The time cost of UIM, compared to traditional integer linear programming computed in CPU.

models. Each binary-value Ising model follows the same updating process as a standard Ising model. This method fits multi-level-value COPs into binary-value UIM with minimal modification. However, random perturbations are applied to all spins evenly in the updating iterations, and chances are that more than one spin remains +1 for an n -spin vertex after perturbation. It results in conflicts and additional filtering to reach a meaningful solution. This work presents a single-selecting rule (Supplementary Note 16). It includes a step to ensure that only one spin remains +1 for the n -spin vertex, that is, the n graphs keep synchronized. The graph aggregation operation remains the same as the standard Ising model. The single-selecting rule enables multi-level-value COP to be well-fitted in UIM. Different from previous work, in this work, the Hamiltonian of a given graph (constraints are not shown in the equation) is:

$$H = \sum_{u \in E} \sum_{i=1}^n \sigma_{u,i} \sigma_{v,i} \quad (4)$$

The term $\sigma_{u,i}$ denotes whether vertex u is coloured by colour i . H denotes the remaining conflicts in the whole graph. During the

updating epochs of vertex v , the local Hamiltonian corresponding to v refers to the following equation:

$$H(\sigma_v) = \sum_{i=1}^n \sigma_{v,i} \left(\sum_{u \in E} \sigma_{u,i} \right) \quad (5)$$

Spin $\sigma_{v,i}$ aggregates the values of its neighbouring vertices. For example, given the seven-colour problem, one aggregated result of an n -spin vertex at some epoch is {7, 5, 2, 1, 0, 0, 0}. The non-zero elements ('7, 5, 2, 1' in this example) imply the neighbour vertices already have the corresponding colour(s). Graph colouring is looking for the neighbouring vertices with different colours. Thus, the colour of this vertex in the next epoch should be among the colours corresponding to those three zero (or least aggregation result) elements. If there exists more than one minor aggregation result, we randomly choose one colour among them as the updated vertex colour. This step acts as the random perturbation in updating iterations. The convergence is validated in Fig. 6b,c (see Supplementary Note 15 for the detailed benchmark). Figure 6b,c shows the evolution of vertex colours and conflict numbers with the single-selecting rule, respectively. With the

non-zero submatrix size set to 4×4 , Fig. 6d–f benchmarks the computation solving graph colouring using the COLOR dataset³⁷. On the three typical graphs ‘4-Fullns_4’, ‘3-Insertions_5’ and ‘4-Insertions_4’, this work uses $3.4\text{--}7.0\times$ fewer memories with CG-CSR (Fig. 6d). The proposed hardware scheme demonstrates $442\text{--}1,451\times$ speedup compared to GPU³⁸ (Fig. 6e), and the energy consumption also outperforms GPU by $4.1 \times 10^5\text{--}6.0 \times 10^5$ times (Fig. 6f).

EDA accelerating applications with UIM

The proposed CIM-based UIM with binary-value and multi-level-value COP mapping techniques allow the efficient solution of real-world NP-hard COPs. Here, we offer a practical EDA COP problem solved by UIM as an example. MPL layout decomposition is essential for 45 nm and sub-45 nm CMOS technologies^{34,39}. MPL decomposes a layout feature into two or more masks so that each cover’s pattern satisfies the minimum colouring spacing. This technique helps increase the yield of chip fabrication. MPL is a graph colouring problem. Patterns in layout correspond to vertices in a graph. A mask is a colour in the graph colouring problem (Fig. 6g). We benchmark the MPL graph colouring problem with three and four colours in the ISCAS dataset⁴⁰, a practical integrated circuit layout dataset. Figure 6h shows an example of the colouring results. This work increases effective data density by $\sim 100\%$ due to highly parallel CIM operations. The proposed Ising machine achieved $396\text{--}65,550$ times speedup towards convergence (Fig. 6i), compared to the integer linear programming algorithm, a traditional MPL solving method running on a CPU.

Conclusions

We have reported a UIM that accelerates processing Ising graphs using interaction-centric storage and CIM hardware. The architecture is flexible because modified versions of the adjacency matrix of graphs are stored, which supports arbitrary Ising graphs, and the partition granularity and memory utility are adjustable dynamically after chip fabrication. Therefore, the physical integration of memory cells does not affect the problem mapping. CG-CSR compresses the Ising graph adjacency matrices. Because only non-zero submatrices in the adjacency matrix are retained, this scheme is well-suited to using CIM, which can process the non-zero submatrices with high parallelism. The Ising graphs require no repetitive access once written into CIM.

Since the sparsity of the Ising graph is usually high in real-world applications, this scheme boosts energy efficiency and computing speed in various applications. First, its memory cost mostly depends on the number of edges. Generally, the memory cost increases much slower than the number of vertices in very sparse graphs. Therefore, this scheme is more scalable than the existing all-to-all connection schemes of UIMs. Second, the routing complexity and computation complexity both decreases. Although we take the simulated annealing algorithm in this work as a typical example, other heuristic algorithms that involve graph operations exist and these could be adapted for execution and acceleration on the proposed hardware. Deploying more efficient algorithms on our CIM-based UIM provides an opportunity for future work.

Methods

Measurement

We used an Agilent B1500A to measure the characteristics of the fabricated RRAM device. During d.c. operation, an external voltage was applied to BL (Bit-Line) with corresponding WL (Word-Line) and SL (Source-Line) connected to the external power rails ‘Vdd’ (Voltage Drain-Drain) and ‘Gnd’ (Ground), respectively. The sensing operation was conducted by applying a bias voltage of 100 mV to an RRAM device and recording the corresponding device current. The conductance data in Fig. 4c were measured after the set–reset process of 60 devices. The conductance mapping shown in Fig. 5c was collected from 128×128 array.

To measure the characteristics of the integrated RRAM chip, we built a testing system with custom-designed printed circuit boards (PCB) and Xilinx VC707 FPGA development board. The FPGA uses digital logic to control the programming process and the PCB-level data flow via the FPGA Mezzanine Card interface. With the VC707 FPGA development board, on-board digital-to-analogue converters (model number DAC8168) and analogue-to-digital converters (ADCs) (model number MCP33131/21/11) can automatically program and read RRAM cells in large arrays. The programming yield on an RRAM array ranges from 99.7 to 100.0%.

Implementation of UIM demonstration

A closed-loop CIM-based UIM system was developed using a 128×128 1T1R RRAM macro, a custom-designed PCB and a Xilinx VC707 FPGA evaluation board. The custom PCB included drivers for programming RRAM and ADCs for data readout. CIM VMM operation was done in an RRAM macro while addressing, and other digital logics were deployed on FPGA. The demonstration used on-board programming of 16,384 devices. To demonstrate the computing process in real-time, an on-board liquid crystal display was included to present the cut number at each iteration. We used MicroBlaze on VC707 as a soft processing core to receive hardware processing results and pass them to the display via the advanced extensible interface bus. The computation was recorded as a video clip shown in Supplementary Note 19.

Data availability

Source data are available via Zenodo at <https://doi.org/10.5281/zenodo.10686168> (ref. 41). Other data that support the findings of this study are available from the corresponding authors upon reasonable request.

Code availability

The core source code that implements the full-FPGA version of the UIM architecture is available via Zenodo at <https://doi.org/10.5281/zenodo.10686168> (ref. 41). Other codes are available from the corresponding authors upon reasonable request.

References

- Karp, R. M. *Reducibility Among Combinatorial Problems* (Springer, 2010).
- Aadit, N. A. et al. Massively parallel probabilistic computing with sparse Ising machines. *Nat. Electron.* **5**, 460–468 (2022).
- Schuetz, M. J., Brubaker, J. K. & Katzgraber, H. G. Combinatorial optimization with physics-inspired graph neural networks. *Nat. Mach. Intell.* **4**, 367–377 (2022).
- Korte, B. H., Vygen, J., Korte, B. & Vygen, J. *Combinatorial Optimization* Vol. 1 (Springer, 2011).
- Lin, S. & Kernighan, B. W. An effective heuristic algorithm for the traveling-salesman problem. *Oper. Res.* **21**, 498–516 (1973).
- Kokash, N. *An Introduction to Heuristic Algorithms* (Univ. Trento, 2005).
- Tatsumura, K., Yamasaki, M. & Goto, H. Scaling out Ising machines using a multi-chip architecture for simulated bifurcation. *Nat. Electron.* **4**, 208–217 (2021).
- Mohseni, N., McMahon, P. L. & Byrnes, T. Ising machines as hardware solvers of combinatorial optimization problems. *Nat. Rev. Phys.* **4**, 363–379 (2022).
- Hu, F., Wang, B.-N., Wang, N. & Wang, C. Quantum machine learning with D-wave quantum computer. *Quantum Eng.* **1**, e12 (2019).
- Johnson, M. W. et al. Quantum annealing with manufactured spins. *Nature* **473**, 194–198 (2011).
- Inagaki, T. et al. A coherent Ising machine for 2000-node optimization problems. *Science* **354**, 603–606 (2016).

12. Dutta, S. et al. An Ising Hamiltonian solver based on coupled stochastic phase-transition nano-oscillators. *Nat. Electron.* **4**, 502–512 (2021).
13. Moy, W. et al. A 1,968-node coupled ring oscillator circuit for combinatorial optimization problem solving. *Nat. Electron.* **5**, 310–317 (2022).
14. Takemoto, T., Hayashi, M., Yoshimura, C. & Yamaoka, M. 2.6 A 2×30k-spin multichip scalable annealing processor based on a processing-in-memory approach for solving large-scale combinatorial optimization problems. In *IEEE International Solid-State Circuits Conference (ISSCC)* 52–54 (IEEE, 2019).
15. Takemoto, T. et al. 4.6 a 144kb annealing system composed of 9×16kb annealing processor chips with scalable chip-to-chip connections for large-scale combinatorial optimization problems. In *IEEE International Solid-State Circuits Conference (ISSCC)* Vol. 64, 64–66 (IEEE, 2021).
16. Yamamoto, K. et al. 7.3 statica: a 512-spin 0.25 m-weight full-digital annealing processor with a near-memory all-spin-updates-at-once architecture for combinatorial optimization with complete spin-spin interactions. In *IEEE International Solid-State Circuits Conference (ISSCC)* 138–140 (IEEE, 2020).
17. Chou, J., Bramhavar, S., Ghosh, S. & Herzog, W. Analog coupled oscillator based weighted Ising machine. *Sci. Rep.* **9**, 14786 (2019).
18. Su, Y., Kim, H. & Kim, B. CIM-spin: a scalable CMOS annealing processor with digital in-memory spin operators and register spins for combinatorial optimization problems. *IEEE J. Solid-State Circuits* **57**, 2263–2273 (2022).
19. Su, Y., Mu, J., Kim, H. & Kim, B. A scalable CMOS Ising computer featuring sparse and reconfigurable spin interconnects for solving combinatorial optimization problems. *IEEE J. Solid-State Circuits* **57**, 858–868 (2022).
20. Wang, T. & Roychowdhury, J. OIM: oscillator-based Ising machines for solving combinatorial optimisation problems. In *Unconventional Computation and Natural Computation: 18th International Conference, UCNC 2019* Vol. 18, 232–256 (Springer International Publishing, 2019).
21. Wang, T., Wu, L. & Roychowdhury, J. New computational results and hardware prototypes for oscillator-based Ising machines. In *Proc. 56th Design Automation Conference (DAC)* 1–2 (ACM, 2019).
22. Tatsumura, K., Dixon, A. R. & Goto, H. FPGA-based simulated bifurcation machine. In *Proc. International Conference on Field Programmable Logic and Applications (FPL)* 59–66 (IEEE, 2019).
23. Yamamoto, K. et al. A time-division multiplexing Ising machine on FPGAs. In *Proc. International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies* 1–6 (ACM, 2017).
24. Cook, C., Jin, W. & Tan, S. X.-D. GPU-based Ising computing for solving balanced min-cut graph partitioning problem. Preprint at <https://arxiv.org/abs/1908.00210> (2019).
25. Cook, C., Zhao, H., Sato, T., Hiromoto, M. & Tan, S. X.-D. GPU-based Ising computing for solving max-cut combinatorial optimization problems. *Integration* **69**, 335–344 (2019).
26. Verma, N. et al. In-memory computing: advances and prospects. *IEEE Solid-State Circuits Mag.* **11**, 43–55 (2019).
27. Roy, K., Chakraborty, I., Ali, M., Ankit, A. & Agrawal, A. In-memory computing in emerging memory technologies for machine learning: an overview. In *Proc. 57th Design Automation Conference (DAC)* 1–6 (IEEE, 2020).
28. Yan, B. et al. Resistive memory-based in-memory computing: from device and large-scale integration system perspectives. *Adv. Intell. Syst.* **1**, 1900068 (2019).
29. Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R. & Eleftheriou, E. Memory devices and applications for in-memory computing. *Nat. Nanotechnol.* **15**, 529–544 (2020).
30. Jiang, M., Shan, K., He, C. & Li, C. Efficient combinatorial optimization by quantum-inspired parallel annealing in analogue memristor crossbar. *Nat. Commun.* **14**, 5927 (2023).
31. Barahona, F., Grötschel, M., Jünger, M. & Reinelt, G. An application of combinatorial optimization to statistical physics and circuit layout design. *Oper. Res.* **36**, 493–513 (1988).
32. Goemans, M. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. Assoc. Comput. Mach.* **42**, 330–343 (1995).
33. Coudert, O. Exact coloring of real-life graphs is easy. In *Proc. 34th Design Automation Conference (DAC)* 121–126 (ACM, 1997).
34. Kahng, A. B., Park, C.-H., Xu, X. & Yao, H. Layout decomposition approaches for double patterning lithography. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **29**, 939–952 (2010).
35. Ciesielski, M. J., Yang, S. & Perkowski, M. A. Multiple-valued Boolean minimization based on graph coloring. In *Proc. International Conference on Computer Design: VLSI in Computers and Processors* 262–263 (IEEE, 1989).
36. Smith, M. D., Ramsey, N. & Holloway, G. A generalized algorithm for graph-coloring register allocation. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation* 277–288 (ACM, 2004).
37. Johnson, D. S. & Trick, M. A. *Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge* Vol. 26 (American Mathematical Society, 1996).
38. Zhang, K., Qiu, M., Li, L. & Liu, X. Accelerating genetic algorithm for solving graph coloring problem based on CUDA architecture. In *Bio-Inspired Computing—Theories and Applications* 578–584 (Springer, 2014).
39. Ma, Y., Zeng, X. & Yu, B. Methodologies for layout decomposition and mask optimization: a systematic review. In *Proc. International Conference on Very Large Scale Integration (VLSI-SoC)* 1–6 (IEEE, 2017).
40. Li, W. et al. OpenMPL: an open-source layout decomposer. *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* **40**, 2331–2344 (2020).
41. Yue, W. Scalable universal Ising machine enabled by interaction-centric storage and compute-in-memory technology. *Zenodo* <https://doi.org/10.5281/zenodo.10686168> (2024).

Acknowledgements

This work was supported by the National Key R&D Programme of China (grant no. 2023YFB4502200), National Natural Science Foundation of China (grant nos 61925401, 92064004, 61927901, 8206100486, 92164302 and 92364102), Beijing Natural Science Foundation (grant no. L234026) and the 111 Project (grant no. B18001). Fund grant nos 2023YFB4502200, 61925401, 92064004, 92164302 and L234026 were awarded to Yuchao Yang. Fund grant no. 61927901 was awarded to R.H. Fund grant no. 8206100486 was awarded to Y.T. Fund grant no. 92364102 was awarded to B.Y.

Author contributions

B.Y. and Yuchao Yang directed the research. W.Y. and B.Y. conceived the idea and planned the study. W.Y., Z.J. and B.Y. designed the RRAM chip. W.Y. and B.Y. designed the UIM architecture. W.Y. implemented the closed-loop CIM-based UIM system. Y.L. extended the UIM to the EDA application. T.Z., Yuxiang Yang, Z.Y., Y.W., W.B., K.Z., J.K., R.H. and Yuchao Yang developed the RRAM integration processes. T.Z., W.Y., Yuxiang Yang, Z.Y., Y.W. and Y.T. optimized the RRAM device. W.Y. and K.W. developed the video demonstration. K.W. optimized the testing platform. All authors reviewed and edited the paper.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41928-024-01228-7>.

Correspondence and requests for materials should be addressed to Bonan Yan or Yuchao Yang.

Peer review information *Nature Electronics* thanks Bin Gao, Luke Theogarajan and Masanao Yamaoka for their contribution to the peer review of this work.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2024